Reporte de Práctica 4: Comunicación bidireccional UART entre FPGA DE10-Lite y computadora usando MATLAB y ESP32

Santiago López Aldana - 185008
Mario Cardeña García - 180220
Universidad de Las Americas Puebla (UDLAP)
Laboratorio de Sistemas Embebidos

30 de octubre de 2026

Resumen

En esta práctica se desarrolla un sistema de comunicación serial bidireccional mediante el protocolo UART entre la tarjeta FPGA DE10-Lite y una computadora. Para lograr el enlace físico, se utiliza un módulo ESP32 configurado como puente USB-UART. Para ello, se implementa en Verilog los módulos de recepción ($uart_rx$) y transmisión ($uart_tx$), calculando los ciclos de reloj necesarios para sincronizar la comunicación a 9600 baudios. Del lado de la computadora, empleamos MATLAB para el envío y lectura de datos por el puerto serial. Al final, integramos todos los módulos por donde la FPGA recibe un byte, lo procesa, y lo transmite de regreso, comprobando mediante las pruebas finales el correcto funcionamiento de toda la arquitectura de comunicación.

Objetivo

Diseñar e implementar un sistema de comunicación serial bidireccional entre una tarjeta FPGA DE10-Lite y una computadora mediante el protocolo UART. El sistema incorpora un módulo ESP32 que funciona como puente USB-UART para enlazar físicamente ambos dispositivos, además de emplear scripts en MATLAB desde la PC para el envío y recepción de la información. Finalmente, la lógica integrada en la FPGA permite recibir, procesar y transmitir de regreso los datos operando a una velocidad de 9600 baudios.

Estado del Arte

La comunicación serial Universal Asynchronous Receiver-Transmitter (UART) es un método de transferencia de datos asíncrono en el cual no se comparte una señal de reloj entre el transmisor y el receptor. Para lograr el enlace, ambos dispositivos deben configurarse con la misma velocidad de transmisión o *baud rate*. La información se envía a través de tramas de datos, las cuales típicamente inician con 1 bit de inicio activo en bajo, seguido de 8 bits de datos y finalizan con 1 bit de paro en alto (8N1). Además, es posible comprobar la integridad de los datos recibidos al añadir un bit extra que actúa como medidor de

paridad, que realiza la operación XOR a cada bit del contenido. El estado intermedio de inactividad entre la transferencia de datos se conoce como IDLE, el cual se representa con uma cadena de bits en alto. Para establecer una comunicación bidireccional se requiere emplear dos líneas principales: TX para transmisión y RX para recepción, interconectadas de manera cruzada entre los dispositivos, de modo que el bit menos significativo del contenido se envíe primero seguido de los bits de mayor significancia [2].

Dado que la FPGA posee un oscilador de reloj preciso de 50 MHz, es necesario adaptar el *tick* a la velocidad estándar de la comunicación serial, que comúnmente es una transmisión de 9600 baudios. Así, cada bit se transfiere al contarse aproximadamente 5208 ciclos de reloj, es decir, cada $104{,}1\overline{6}\mu s$. De esta manera, es indispensable contar en esta ventana de tiempo cada bit de manera correcta, por lo que lo más seguro es esperar a la mitad del tiempo del bit transmitido para verificar su valor y así no leerlo en el momento exacto de cambio de estado `posedge`.

Por otro lado, a nivel de hardware, todos los pines GPIO de la FPGA DE10-Lite operan también como pins rx y tx [1]. Así, se emplea un ESP32 configurado como puente USB-UART en el entorno de Arduino IDE para conectarlo directamente a los pines de la FPGA, encargándose de recibir la información de la PC por USB, reenviarla por UART y regresar la respuesta a la computadora.

Planteamiento del Experimento

Se plantea utilizar un microcontrolador ESP32 como puente USB-UART por el cual la computadora ingrese un número de 8 bits que sea transferido a un DE10-Lite que realice una operación que le sume 1 al valor ingresado y lo devuelva a la computadora para observar el resultado por medio de protocolo UART sin bit de paridad. De este modo, es necesario emplear el programa Arduino IDE y MATLAB para para establecer comunicación en serie con el ESP32 y leer los resultados, ya sea en el monitor en serie de Arduino o en la consola de MATLAB. Sin embargo, en caso de usar el Arduino IDE es necesario considerar que su monitor considera las entradas como los caracteres de tabla ASCII en lugar de valores numéricos literales y conjuntos.

Jerarquía

- `uart_rx`: Verifica constantemente los bits recibidos, los cuales se encuentran en estado ILDE con un valor de 1. No es hasta que detecta un cambio de estado a 0 (bit de inicio) y espera la finalización de ese bit, que inicia a almacenar cada bit recibido a la mitad de su periodo al contar cada ciclo de reloj hasta guardar los 8 bits de datos.

- `uart_tx`: Transfiere constantemente un estado ILDE hasta que recibe una señal de inicio para enviar los 8 bits de datos junto al bit de entrada (0) y de paro (1). Posteriormente, transmite cada bit con duración de 5208 ciclos de reloj. Cuando transmite esos 10 bits vuelve al estado de espera ILDE.

- `top_uart_de10lite`: Emplea los módulos `uart_rx` y `uart_tx` para recibir un byte de datos y sumarle a ese valor 1. Después, transfiere ese nuevo bit. Además, cuenta con una opción de `reset` compartida con los otros dos módulos para detener la transmición y resetear los bits del contenido.

Además de estos módulos para el control del FPGA, se cuenta también con el programa `arduino_esp32_bridge` para habilitar la comunicación en serie con el ESP32. Así como el programa `bidireccional_matlab` para realizar la comunicación en su consola.

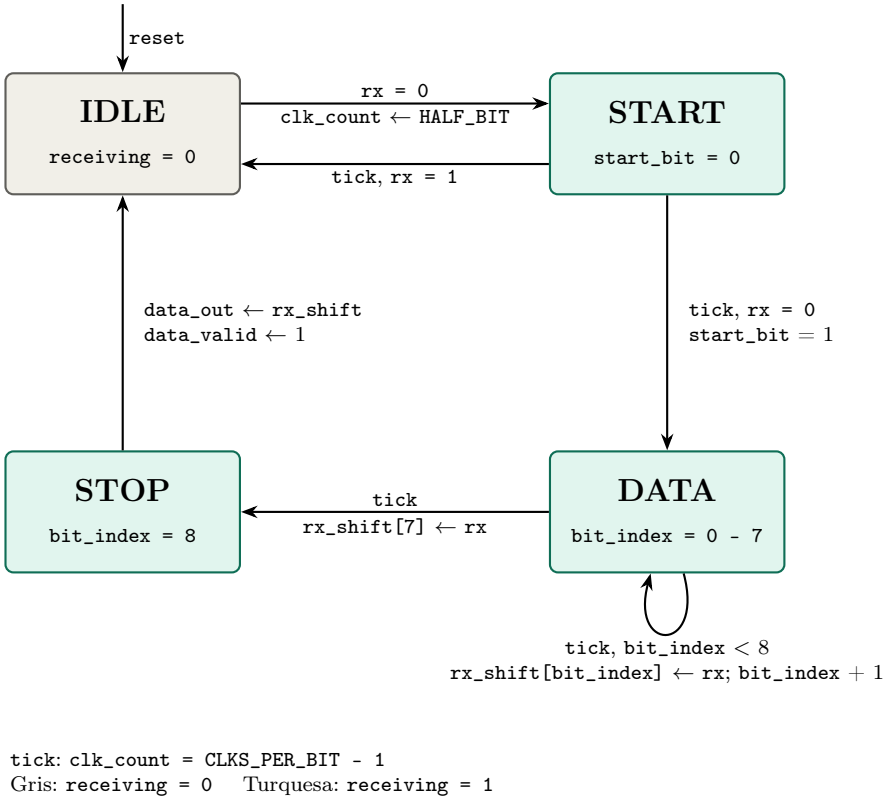


Figura 1: Máquina de estados del receptor `uart_rx`.

En la Figura 1 se ilustra la máquina de estados finitos que controla el funcionamiento del módulo receptor `uart_rx`. El sistema arranca en el estado IDLE esperando a que la señal de entrada `rx` baje a cero para detectar el bit de inicio. Tras confirmar este bit en el estado START, transita al estado DATA donde muestrea secuencialmente los 8 bits que conforman el mensaje, en caso contrario donde el bit de inicio no se valida, vuelve al estado IDLE. En este caso, no se comprueba el bit de finalización, solo se espera a que hubieran transcurrido los 8 bits de datos para detener el proceso y cambiar al estado STOP, aunque en el código no se expresa este estado como tal.

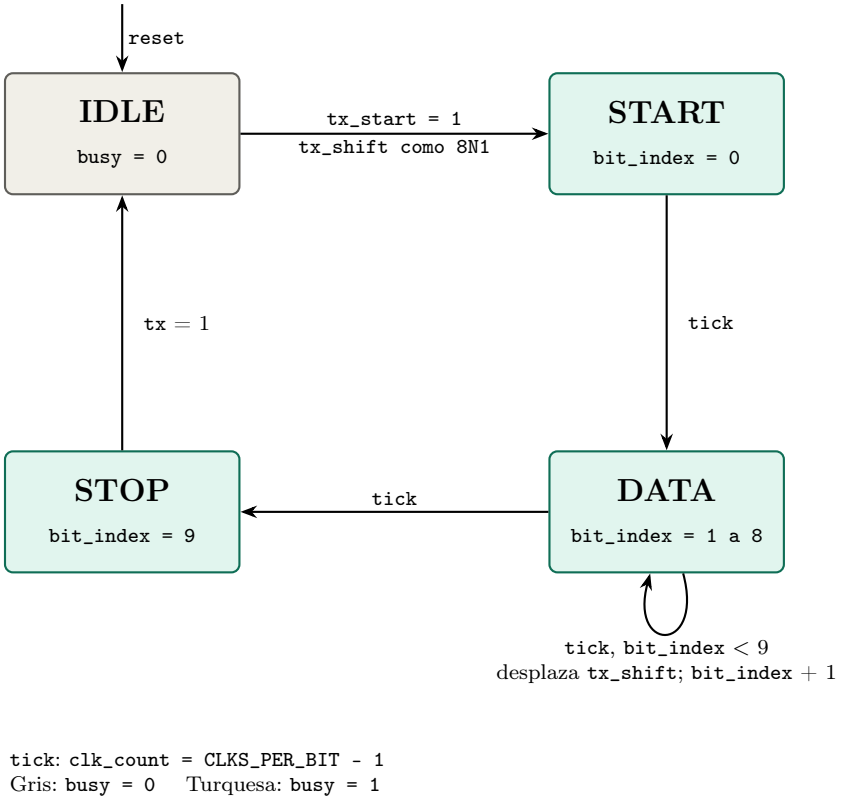


Figura 2: Máquina de estados del transmisor `uart_tx`.

En la Figura 2 se detalla la máquina de estados del módulo transmisor `uart_tx`. El proceso inicia en el estado IDLE, donde espera la señal `tx_start` para activar la bandera `busy`, cargar el dato a transmitir y avanzar al estado START. Posteriormente, transita al estado DATA, encargado de enviar los 10 bits de información desplazando secuencialmente el registro en cada pulso de sincronía o *tick*. Una vez completado el envío, regresa al estado IDLE, bajando la señal `busy` para indicar que el módulo está libre para una nueva transmisión, en el código no se refleja un estado STOP como tal. Los módulos en verilog empleados no siguen una estructura de de máquina de estados tradicional al no usar `localparam` para definir los estados o definir exactamente la secuencia de estos con varios bloques `always`.

Materiales y Equipo

- **Hardware:** Tarjeta de desarrollo Terasic DE10-Lite (FPGA Intel MAX 10), equipo de cómputo, módulo ESP32, cable USB Blaster para la programación de la FPGA, cable USB para el ESP32 y cables Dupont para la interconexión física cruzada.
- **Software:** Entorno Active-HDL para la simulación funcional, Quartus Prime para la compilación, asignación de pines y programación de la FPGA, MATLAB (con soporte de comunicación serial) para la ejecución de los scripts de control, y Arduino IDE para configurar el puente transparente USB-UART en el ESP32.

Diseño Experimental

Por medio de estos equipos, contextos y necesidades discutidas, se programa el código fuente que se encuentra en el link de a continuación:

Repositorio de GitHub – uart_comunicacion_bidireccional

Existe un paso crítico al leer los datos recibidos tras detectar el bit de inicio cuando la línea pasa de 1 a 0, pues es posible que se almacene dicho bit de inicio en lugar del primer bit de dato D0. Por esta razón, es necesario agregar en el código del módulo `uart_rx` un segmento que evalúe el tiempo transcurrido desde la transición de IDLE al bit de inicio y espere a que dicho bit transcurra, para comenzar a recibir desde la mitad de D0 y no desde el bit de inicio.

Por ejemplo, cuando se envían los bits de datos `00001010`, que corresponden a 10 en decimal, la transferencia ocurre de la siguiente manera:

```
...11111111_0 01010000 1_11111111...
```

Es decir, tras el IDLE aparece el bit de inicio en 0, seguido de los datos desde el bit menos significativo (D0) hasta el más significativo (D7). Si no se espera a que transcurra el bit de inicio, se captura:

```
0010100
```

de modo que se pierde D7 por *overflow*. Este valor corresponde a `00010100` = 20 en decimal. Por lo tanto, el valor se multiplica por dos, pues se desplaza un bit a la izquierda como si se aplicara la operación $\ll 1$. El resultado que recibe la computadora es 21. Ahora bien, si este error se corrige pero se emplea el monitor en serie sin convertir los caracteres ASCII de dos o tres dígitos para la representación de un valor decimal, se le sumará 1 a cada valor posicional representado por el símbolo numérico ASCII. Es decir, se envia individualmente cada caracter y se le suma uno para devolver el siguiente valor de la tabla.

Una vez arreglado el problema del bit de inicio, al agregar el paso equivalente al estado START de la Figura 1 que valida el bit de inicio, se obtiene lo siguiente al probarlo en el entorno de Quartus:

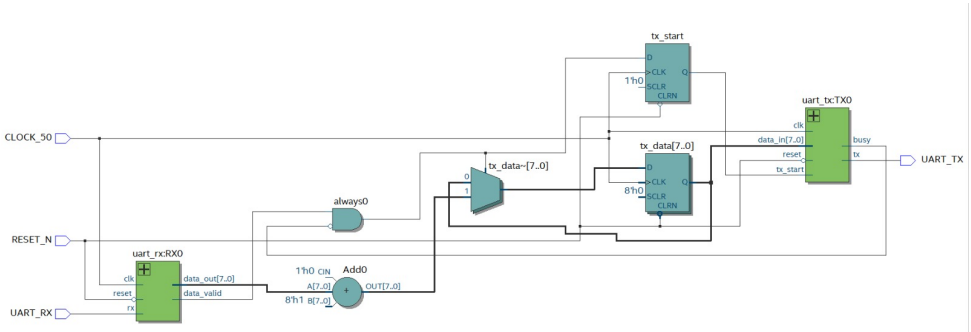


Figura 3: Esquema RTL de Quartus del módulo `top_uart_de10lite`.

En la Figura 3 se presenta el esquema RTL generado por el programa para el módulo principal `top_uart_de10lite`. Aquí se aprecia el flujo físico de la información: la señal ingresa por la izquierda al bloque receptor `uart_rx`, el cual extrae el byte y lo pasa a una lógica combinacional que le suma una unidad. Finalmente, este dato procesado se carga en el transmisor `uart_tx` de la derecha para ser enviado de regreso por el pin de salida, sincronizando todo con el reloj central de 50 MHz de la tarjeta.

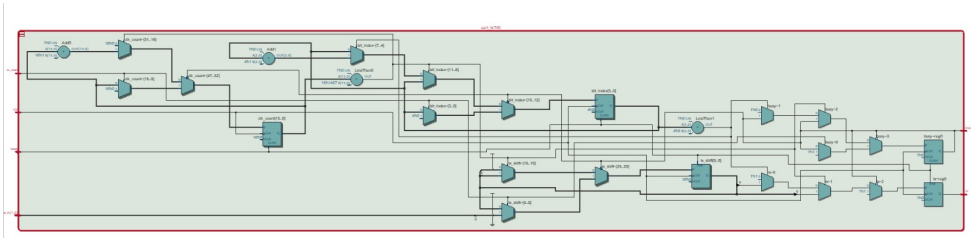


Figura 4: Esquema RTL de Quartus del módulo `uart_tx`.

Por su parte, en la Figura 4 se observa el esquema RTL del módulo transmisor `uart_tx`. En este diagrama se aprecia la estructura interna que se muestra en la Figura 2, en la cual se utilizan sumadores, flip-flops y multiplexores para construirla. Se toma el byte por la entrada para transmitirlo en el formato especificado anteriormente bit por bit a través de `tx`, mientras el parámetro `busy` este activo.

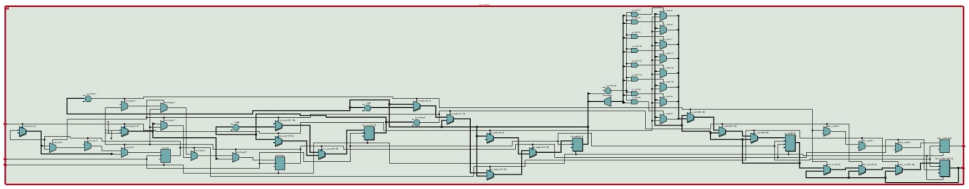


Figura 5: Esquema RTL de Quartus del módulo `uart_rx`.

Asímismo,en la Figura 5 se observa el RTL del módulo receptor `uart_rx` construida por una red compleja de los mismos materiales. En este caso, `rx` recibe bit por bit y se almacena tras detectar el bit de inicio, al finalizar levanta la bandera `data_valid`.

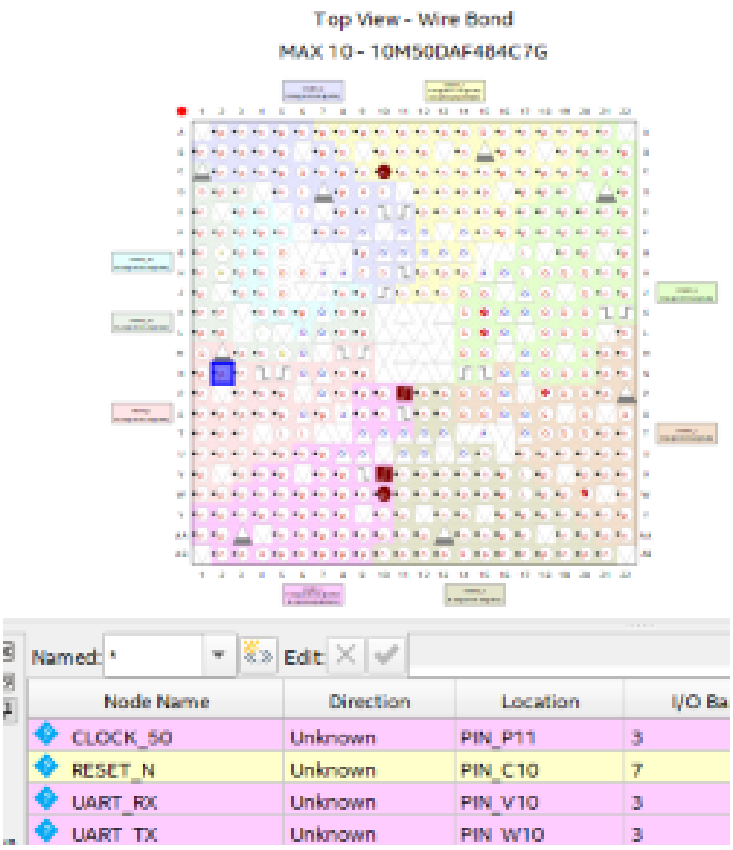


Figura 6: Vista del Pinplanner de Quartus

La Figura 6 muestra la configuración del hardware FPGA en la tarjeta DE10-Lite, donde se emplean dos pins GPIO para establecer el pin rx, el cual va conectado al pin tx del ESP32, y el pin tx, el cual va conectado al pin rx del ESP32. Asímismo, se configura un reloj de 50MHz y un interruptor que funciona como `reset`.

Casos de Estudio o Pruebas

```
Ingrese un numero entre 0 y 255: 1
Enviado: 1
Respuesta de FPGA: 2
Ingrese un numero entre 0 y 255: 1
Enviado: 1
No se recibio respuesta
```

Figura 7: Prueba de envíos consecutivos en la terminal de MATLAB del interruptor `reset`.

Una vez compilados y ejecutados los códigos en el ESP32 y en el FPGA, se emplea el entorno de Matlab para comunicarse con el ESP32, que a su vez se comunica con el FPGA por medio de los pins 16 y 17. De esta manera, se probó la comunicación al ingresar números en el rango permitido de 8 bits y comprobar el interruptor de `reset` como se muestra en la Figura 7. Además, se probó la conexión de los cables entre los pins rx y tx de tanto el FPGA, así como del ESP32, demostrando que una conexión floja puede causar la devolución de resultados errados.

Análisis de Resultados

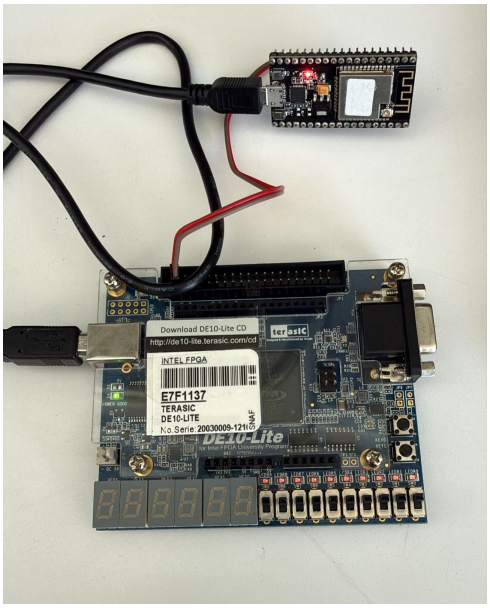


Figura 8: Vista del FPGA DE10-Lite y ESP32 empleados

```
Respuesta de FPGA: 18
Ingrese un numero entre 0 y 255: 18
Enviado: 18
Respuesta de FPGA: 19
Ingrese un numero entre 0 y 255: 19
Enviado: 19
Respuesta de FPGA: 20
Ingrese un numero entre 0 y 255: 20
Enviado: 20
Respuesta de FPGA: 21
Ingrese un numero entre 0 y 255: 21
Enviado: 21
Respuesta de FPGA: 22
Ingrese un numero entre 0 y 255: 22
Enviado: 22
Respuesta de FPGA: 23
Ingrese un numero entre 0 y 255: 23
Enviado: 23
Respuesta de FPGA: 24
Ingrese un numero entre 0 y 255: 24
Enviado: 24
Respuesta de FPGA: 25
Ingrese un numero entre 0 y 255: 25
Enviado: 25
Respuesta de FPGA: 26
Ingrese un numero entre 0 y 255: 100
Enviado: 100
Respuesta de FPGA: 101
Ingrese un numero entre 0 y 255: 101
Enviado: 101
Respuesta de FPGA: 102
Ingrese un numero entre 0 y 255: 0
Enviado: 0
Respuesta de FPGA: 1
Ingrese un numero entre 0 y 255: 255
Enviado: 255
Respuesta de FPGA: 0
Ingrese un numero entre 0 y 255: 98
Enviado: 98
Respuesta de FPGA: 99
Ingrese un numero entre 0 y 255: 134
Enviado: 134
Respuesta de FPGA: 135
```

Figura 9: Pruebas de comunicación bidireccional en Matlab implementando la función $A + 1$.

En la Figura 9 se detalla las pruebas de la comunicación bidireccional realizadas. Como se aprecia en la captura de la terminal, la FPGA recibe exitosamente los datos, sumando una unidad al valor enviado, tal como se observa en los casos del 18 o el 101. También, responde de manera correcta ante los casos límite: con 0 se retorna un 1, mientras que con el valor máximo de 255, se genera un desbordamiento que devuelve un 0, validando los 8 bits del registro.

```
Enviado: 20
Respuesta de FPGA: 35777.1
------------------------------------------------
Ingrese un numero entre 0 y 255: 21
Enviado: 21
Respuesta de FPGA: 42439.2
------------------------------------------------
Ingrese un numero entre 0 y 255: 22
Enviado: 22
Respuesta de FPGA: 49943.5
------------------------------------------------
Ingrese un numero entre 0 y 255: 23
Enviado: 23
Respuesta de FPGA: 58350.9
------------------------------------------------
Ingrese un numero entre 0 y 255: 24
Enviado: 24
Respuesta de FPGA: 67723.5
------------------------------------------------
Ingrese un numero entre 0 y 255: 25
Enviado: 25
Respuesta de FPGA: 78125
------------------------------------------------
Ingrese un numero entre 0 y 255: 100
Enviado: 100
Respuesta de FPGA: 10000000
------------------------------------------------
Ingrese un numero entre 0 y 255: 150
Enviado: 150
Respuesta de FPGA: 41335139.4
------------------------------------------------
Ingrese un numero entre 0 y 255: 200
Enviado: 200
Respuesta de FPGA: 113137085
------------------------------------------------
Ingrese un numero entre 0 y 255: 255
Enviado: 255
Respuesta de FPGA: 264783325
------------------------------------------------
Ingrese un numero entre 0 y 255: 0
Enviado: 0
Respuesta de FPGA: 0
------------------------------------------------
```

Figura 10: Pruebas de comunicación bidireccional en Matlab implementando la función de potencia $A^{\frac{7}{2}}$.

Por otro lado, en la Figura 10 se presentan los resultados obtenidos al implementar la función $A^{\frac{7}{2}}$ solicitada para punto extra. Para lograr este escenario se implementó un módulo que implementa una Look-Up Table (LUT) con todos los valores correspondientes a A con longitud de 8 bits, siendo el resultado multiplicado por 10 para obtener un punto decimal del resultado que se guarda en un registro de 32 bits. Sin embargo, puesto que solo se pueden transmitir datos de 8 bits, es necesario realizar una transmisión secuencial de 4 bytes. En Matlab se recogen las 4 transmisiones y divide el resultado por 10 para imprimirlo. Sin embargo, aunque el empleo de la LUT presenta la ventaja de velocidad, sufre el problema de ser una tabla de datos precalculados y limitados, por lo que a la hora

de evaluar otras funciones distintas o más complejas, se vuelve innecesaria a comparación del empleo de un módulo de operaciones más flexible.

Conclusiones

En conclusión, se logra implementar con éxito una comunicación UART bidireccional entre la computadora y la FPGA utilizando el ESP32 como puente. Comprobamos que nuestra lógica procesa y sincroniza correctamente los datos, manejando sin problemas los casos límite para las funciones de suma y potencia al identificar el error de bit de inicio que se coló en los bits registrados. Además, al desarrollar el punto extra, establecimos una LUT y logramos dividir la transmisión de un registro de 32 bits en paquetes de 8 bits.

Referencias

[1] Terasic Technologies, *DE10-Lite User Manual*. [Online]. Available: `https://www.mouser.com/datasheet/2/598/DE10-Lite_User_Manual-1100361.pdf`

[2] Nanda U. & Pattnaik S. K., 2016 , *Universal Asynchronous Receiver and Transmitter (UART)*. Available: `https://doi-org.udlap.idm.oclc.org/10.1109/ICACCS.2016.7586376`